MATLAB

FOUNDATIONS & PRACTICAL EXAMINATION NOTES

A clear, detailed foundation for beginners preparing for college Mathematics-I practical examinations.

Purpose	Build a reliable MATLAB foundation before applying MATLAB to calculus, differential equations, vectors, series, and other Mathematics-I topics.
Audience	A student who understands basic programming but is new to MATLAB.
Method	Concept → MATLAB syntax → example → interpretation → common mistake → practical use → viva.

Core idea: MATLAB is both a programming environment and a mathematical computing system. You should learn what a command does, why it is used, and how its result relates to the mathematics.

Keep MATLAB open while reading this note and type the examples yourself.

Contents

1 MATLAB mental model and terminology

2 Interface: Command Window, Editor, Workspace, Current Folder

3 Your first commands and the semicolon

4 Scripts and comments

5 Variables, assignment, naming rules, and data types

6 Arithmetic, precedence, constants, and built-in functions

7 Vectors, colon operator, indexing, and dimensions

8 Matrices and matrix operations

9 Element-wise operations: .* ./ .^

10 Symbolic variables and expressions

11 Algebra: subs, simplify, expand, factor, collect

12 Equations and inequalities with solve

13 Mathematical functions and anonymous functions

14 Differentiation, integration, and limits

15 Taylor and Maclaurin series

16 Plotting functions correctly

17 if, for, while and practical programming

18 disp and fprintf

19 Standard practical-exam workflow

20 Debugging and common errors

21 Mathematics-I examples

22 Viva questions

23 Practice set and readiness checklist

24 Quick reference

1. MATLAB Mental Model

MATLAB means **MATrix LABoratory**. Its original design was centered on matrix computation, but it now provides numerical computation, symbolic mathematics, visualization, programming, data analysis, and many specialized toolboxes.

For your Mathematics-I practical, think of MATLAB as a bridge between a mathematical statement and an executable procedure.

Mathematics	MATLAB representation	Typical command
x = 3	Store a numerical value	x = 3;
$f(x)=x^2+1$	Symbolic expression	syms x; f=x^2+1;
f(3)	Substitute a value	subs(f,x,3)
$f'(x)$	Derivative	diff(f,x)
$\int f(x)dx$	Integral	int(f,x)
$\lim_{x\to a} f(x)$	Limit	limit(f,x,a)
Graph of f	Function plot	fplot(f)

Two modes you must distinguish:

- **Numerical:** MATLAB works with actual numbers, vectors, matrices, and numerical arrays.
- **Symbolic:** MATLAB keeps symbols such as x and manipulates expressions exactly when the Symbolic Math Toolbox is available.

```
% Numerical
x = 2;
y = x^2 + 1

% Symbolic
syms x
f = x^2 + 1
```

The first block evaluates a number. The second creates a mathematical expression. This distinction becomes important throughout calculus.

2. MATLAB Interface

The interface can vary slightly by release, but the important working areas are consistent.

Area	What it does	How you use it
Command Window	Runs commands immediately	Test syntax and small calculations
Editor	Creates saved .m programs	Write practical-exam solutions
Workspace	Shows current variables	Inspect names, values, and sizes
Current Folder	Shows files MATLAB can access	Save/run scripts correctly
Command History	Stores previous commands	Recover useful commands

The best beginner workflow is: **experiment in Command Window → write the final solution in Editor → run the script → inspect Workspace/output.**

Useful commands:

```
clc          % clear Command Window
clear        % remove variables
close all    % close figure windows
```

Do not confuse them. **clc** changes the display only; **clear** removes variables; **close all** closes graphical windows.

3. Your First Commands

```
2 + 3
10 / 4
5^2
sqrt(49)
sin(pi/2)
```

If an expression is not assigned to a variable, MATLAB normally stores the result in **ans**. This is useful for quick testing, but meaningful variables are better in a final program.

```
a = 2 + 3;
b = sqrt(49);
c = a*b
```

The semicolon **;** suppresses automatic display. It does not stop the calculation.

4. Scripts and Comments

A script is a saved sequence of MATLAB commands, normally stored in an **.m** file. In a practical exam, use a script for anything more than a tiny calculation.

```
% Evaluate a polynomial

clc;
clear;

x = 3;
y = x^3 - 4*x^2 + 7*x - 2;

fprintf('x = %d, y = %.2f\n', x, y);
```

A line beginning with **%** is a comment. Comments do not execute. They explain the purpose of the program or a section of it.

5. Variables and Assignment

MATLAB creates a variable when you assign something to it. You normally do not declare its type first, unlike C.

```
radius = 5;
area = pi*radius^2;
name = 'Jiban';
```

The operator **=** means assignment. It tells MATLAB to evaluate the expression on the right and store the result under the name on the left.

Naming rules:

- Start with a letter.
- Use letters, digits, and underscores after the first character.
- MATLAB is case-sensitive: x and X are different.
- Do not use spaces.
- Use meaningful names.
- Avoid naming a variable after an important MATLAB function, such as sum, mean, max, min, length, plot, or range.

6. Arithmetic and Mathematical Functions

Operator/function	Meaning	Example
+ - * /	Arithmetic	a+b, a*b
^	Power	x^2
sqrt(x)	Square root	sqrt(25)
abs(x)	Absolute value	abs(-7)
exp(x)	e■	exp(2)
log(x)	Natural logarithm	log(5)
log10(x)	Base-10 logarithm	log10(100)

MATLAB follows normal mathematical precedence. Use parentheses whenever they improve clarity.

```
y1 = 2 + 3*4;       % 14
y2 = (2 + 3)*4;     % 20
y3 = 2^3 + 1;       % 9
y4 = 2^(3 + 1);     % 16
```

In mathematical notation, 2x is common. In MATLAB you must write **2*x**. Implicit multiplication is not generally accepted.

7. Trigonometric Functions and Angles

MATLAB's ordinary sin, cos, and tan functions use **radians**.

```
sin(pi/2)     % 1
cos(pi)       % -1

sind(90)      % 1
cosd(180)     % -1
```

If the question gives degrees, use sind/cosd/tand. If it uses radians, use sin/cos/tan.

8. Vectors and the Colon Operator

A row vector is entered using spaces or commas. A column vector uses semicolons between entries.

```
r = [1 2 3 4];
```

```
c = [1; 2; 3; 4];
```

The colon operator generates regularly spaced values:

```
1:5
1:2:9
5:-1:1
```

General form: **start:step:end**. If the step is omitted, MATLAB uses 1.

Indexing is one-based:

```
x = [10 20 30 40 50];

x(1)       % 10
x(3)       % 30
x(2:4)     % 20 30 40
```

This is a major difference from C: MATLAB normally starts indexing at **1**, not 0.

9. Matrices and Dimensions

```
A = [1 2;
     3 4];

B = [5 6;
     7 8];

C = A*B
```

Matrix multiplication gives:

```
C =
    19    22
    43    50
```

If A is m×n and B is n×p, A*B is valid and the result is m×p. Always check dimensions when MATLAB reports a matrix-size error.

10. Matrix vs Element-wise Operations

This is one of the most important MATLAB foundations for a mathematics student.

Syntax	Meaning	Use
A*B	Matrix multiplication	Linear algebra
A.*B	Element-by-element multiplication	Array calculations
A^2	Matrix power	Matrix algebra
A.^2	Square every element	Function values over a vector
A/B	Matrix right division	Linear algebra
A./B	Element-by-element division	Array calculations

Example for plotting a polynomial:

```
x = -5:0.1:5;
y = x.^2 + 2*x + 1;

plot(x,y)
grid on
```

Because x is a vector, **x.^2** squares each element separately. This is why element-wise operators are essential when evaluating functions over a vector.

11. Symbolic Variables and Expressions

Use **syms** when you need a symbolic mathematical variable.

```
syms x
f = x^3 - 4*x^2 + 7*x - 2
```

Now MATLAB knows x as a symbol and can differentiate, integrate, find limits, solve equations, and manipulate the expression.

Numerical vs symbolic:

```
% Numerical
x = 2;
y = x^2 + 1

% Symbolic
syms x
f = x^2 + 1
```

12. Substitution and Algebraic Manipulation

```
syms x

f = x^2 + 2*x + 1;

subs(f,x,3)
expand((x+1)^2)
factor(x^2 - 9)
simplify((x^2-1)/(x-1))
collect(x^3 + 2*x^2 + x,x)
```

Interpret these as mathematical operations: substitution, expansion, factorization, simplification, and collecting terms.

13. Equations and Inequalities

Use **==** to represent equality in a symbolic equation. Do not confuse it with assignment.

```
syms x

eq = x^2 - 5*x + 6 == 0;
solve(eq,x)
```

The roots are 2 and 3.

```
syms x
solve(x^2 - 4 >= 0, x)
```

For inequalities, MATLAB may return a condition or interval representation. Read the result mathematically; the command is a tool, not a replacement for interpretation.

Symbol	Meaning
=	Assignment
==	Equality / equation
> < >= <=	Inequalities

14. Mathematical Functions in MATLAB

There are two useful ways to represent functions for your course.

Symbolic expression — ideal for calculus operations:

```
syms x
f = x^2 + 2*x + 1;
```

Anonymous function — convenient for numerical evaluation and plotting:

```
f = @(x) x.^2 + 2*x + 1;
f(3)
```

Read **@(x)** as “define a function whose input is x.” The expression after it is the formula.

For a vector input, use element-wise operators when the formula must act on each value:

```
f = @(x) x.^2 + 2*x + 1;
x = -3:0.5:3;
y = f(x);
```

15. Differentiation

```
syms x

f = x^3 - 3*x^2 + 2*x;

df  = diff(f,x)
d2f = diff(f,x,2)
```

Mathematically, diff(f,x) gives df/dx and diff(f,x,2) gives d^2f/dx^2.

Always know what you expect before running the command. This lets you detect an incorrect MATLAB expression or wrong variable.

16. Integration

```
syms x

f = x^2;

F = int(f,x)        % indefinite integral
I = int(f,x,0,3)    % definite integral
```

The first is an antiderivative. The second evaluates the definite integral from 0 to 3.

For Mathematics-I practicals, connect the command with the notation you know: ∫f(x)dx and ∫■■f(x)dx.

17. Limits

```
syms x

f = (x^2 - 4)/(x - 2);
L = limit(f,x,2)
```

Direct substitution gives 0/0, but the limit is 4. MATLAB can carry out the symbolic limiting process.

For one-sided limits you may specify a direction when required. Learn the mathematical idea first; then learn the corresponding MATLAB syntax.

18. Taylor and Maclaurin Series

A Taylor series expands a function around a chosen center. A Maclaurin series is centered at zero.

```
syms x
f = exp(x);
```

```
T = taylor(f,x,0,'Order',5)
```

The result is a polynomial approximation to e■ around x = 0. The order controls how many terms are retained.

19. Plotting Functions

Use graphs to inspect domain, range, roots, turning points, monotonicity, asymptotic behavior, and general shape.

Symbolic function with fplot:

```
syms x
f = x^2 - 4;

fplot(f)
grid on
xlabel('x')
ylabel('f(x)')
title('f(x) = x^2 - 4')
```

Numerical vector with plot:

```
x = -5:0.1:5;
y = x.^2 - 4;

plot(x,y)
grid on
xlabel('x')
ylabel('y')
title('y = x^2 - 4')
```

Multiple functions:

```
syms x
f1 = x^2;
f2 = 2*x + 3;

fplot(f1)
hold on
fplot(f2)
hold off

grid on
legend('x^2','2x+3')
```

Useful graph commands:

grid on	show grid
xlabel / ylabel	axis labels
title	graph title
legend	identify curves
xlim / ylim	control visible range
hold on / hold off	draw multiple curves on one axes

20. Conditional Statements

```
x = -3;

if x > 0
    disp('Positive')
elseif x < 0
    disp('Negative')
else
    disp('Zero')
end
```

MATLAB uses **end** to close an if block. This is structurally similar to C, although MATLAB does not use braces.

21. Loops

```
for k = 1:5
    fprintf('k = %d\n', k);
end
```

```
k = 1;

while k <= 5
    fprintf('k = %d\n', k);
    k = k + 1;
end
```

MATLAB supports loops, but many mathematical array operations can be written without explicit loops. This is called vectorization and is an important MATLAB habit.

22. Professional Output

```
x = 3;
y = x^2 + 1;

disp('Calculated value:')
disp(y)

fprintf('For x = %d, y = %.2f\n', x, y);
```

Common fprintf formats: %d for integer, %f for floating-point, %.2f for two decimal places, %s for text, and \n for a new line.

23. Standard Practical-Examination Workflow

Do not begin typing immediately. First convert the question into a mathematical and computational plan.

1. Read the complete question.
2. Identify the object: number, vector, matrix, function, equation, inequality, derivative, integral, limit, or graph.
3. Decide whether numerical or symbolic computation is appropriate.
4. Create the required variables.
5. Write the mathematical expression in MATLAB syntax.
6. Perform the required operation.
7. Display the result clearly.
8. Plot or verify when required.
9. Check dimensions, operators, variable names, and mathematical interpretation.

A reliable practical script template:

```
% Program title / purpose

clc;
clear;
close all;

% Define variables
syms x

% Define mathematical expression
f = x^3 - 3*x + 2;

% Required operation
df = diff(f,x);

% Display
disp('Function:')
disp(f)

disp('Derivative:')
disp(df)

% Plot
fplot(f)
grid on
xlabel('x')
ylabel('f(x)')
title('Given Function')
```

24. Debugging and Common Errors

Error/symptom	Likely cause	What to check
Undefined function or variable	Name not created or misspelled	Spelling, execution order, workspace
Matrix dimensions must agree	Wrong matrix operation	size(A), size(B), and whether .* is intended
Invalid expression	Missing operator or bracket	Use explicit * and matching parentheses
Wrong trig value	Radians/degrees confusion	sin vs sind, etc.
Unexpected old value	Old workspace variable	clear and rerun from top
Script not found	Wrong folder	Current Folder and file name

Error/symptom	Likely cause	What to check
Symbolic result not numerical	Expression still contains symbols	subs or appropriate numerical conversion

The most effective debugging method is to isolate the problem: run the suspicious line alone, inspect variables with size/whos, and compare the MATLAB expression with the original mathematics.

25. Mathematics-I Examples

Example A — Polynomial derivative

Given f(x) = x³ – 4x² + 7x – 2. Find f′(x).

```
clc;
clear;

syms x

f = x^3 - 4*x^2 + 7*x - 2;
df = diff(f,x);

disp('f(x) =')
disp(f)

disp('f''(x) =')
disp(df)
```

Expected mathematics: f′(x) = 3x² – 8x + 7.

Example B — Definite integral

Evaluate ∫■³ x² dx.

```
clc;
clear;

syms x

f = x^2;
I = int(f,x,0,3);

fprintf('Integral = %s\n', char(I));
```

Mathematically, ∫■³x²dx = [x³/3]■³ = 9.

Example C — Limit

Find lim(x→2) (x²–4)/(x–2).

```
syms x

f = (x^2 - 4)/(x - 2);
L = limit(f,x,2)
```

Expected result: 4. This is a good example of why symbolic computation is useful: direct substitution produces an indeterminate form, while the limit process gives a meaningful result.

26. Viva Questions You Should Be Able to Answer

Question	Core answer
What is MATLAB?	A mathematical/scientific computing and programming environment strongly oriented toward matrices and numerical computation.
What does syms do?	Creates symbolic variables for symbolic mathematics.
Difference between = and ==?	= assigns; == represents/tests equality.
Difference between * and .*?	* is matrix multiplication; .* is element-wise multiplication.
Why .^?	To apply powers element-by-element to an array/vector.
Why does indexing start at 1 matter?	MATLAB's standard indexing is one-based, unlike C's typical zero-based indexing.
What is fplot?	A convenient way to plot a function.
What does diff do?	Computes derivatives.
What does int do?	Computes integrals.
What does limit do?	Computes a mathematical limit, especially in symbolic form.
What does clc do?	Clears the Command Window display.
Does clc clear variables?	No. clear is used for variables.
Why use semicolon?	To suppress automatic output while still executing the statement.

27. Foundation Practice Set

Attempt these without copying the solution. For each problem, write a small script and explain what every important command does.

1. Create the vector -5:0.5:5.
2. For x = -5:0.5:5, calculate y = $x^2 + 2x + 1$ using the correct element-wise operation.
3. Create A = [1 2; 3 4] and find its transpose.
4. Calculate AB for A = [1 2;3 4] and B = [5 6;7 8].
5. Define f(x) = $x^3 – 4x^2 + 7x – 2$ symbolically.
6. Find f(3) using subs.
7. Solve $x^2 – 5x + 6 = 0$.
8. Find the first and second derivatives of $x^3 – 3x^2 + 2x$.
9. Evaluate ∫■$^3x^2dx$.
10. Find lim(x→2)(x^2–4)/(x–2).
11. Find a Maclaurin polynomial for e■ using taylor.
12. Plot $y = x^2 – 4$ over a sensible interval and label it.
13. Write a positive/negative/zero program using if.
14. Print the squares from 1 to 10 using a for loop.
15. Write one complete program that defines a symbolic function, differentiates it, displays the derivative, and plots the function.

28. Readiness Checklist

You are ready to move into the Mathematics-I units when you can do these without constantly searching for syntax:

- Create and manipulate scalar variables.
- Write mathematical expressions using explicit MATLAB syntax.
- Use built-in mathematical and trigonometric functions correctly.
- Create row vectors, column vectors, and ranges.
- Index vectors and matrices correctly.
- Explain dimensions and matrix multiplication.
- Know when to use *, .* , ^, and .^.
- Create symbolic variables and expressions.
- Use subs, simplify, expand, factor, and collect.
- Solve equations and inequalities.
- Use symbolic functions for derivatives, integrals, and limits.
- Create Taylor/Maclaurin approximations.
- Plot functions with labels, titles, grids, and legends.
- Write if, for, and while structures.
- Produce clean practical-exam output.
- Read MATLAB errors and identify the likely cause.

29. Quick Reference

Task	Syntax
Clear screen / variables / figures	clc; clear; close all;
Comment	% comment
Symbolic variable	syms x
Substitution	subs(f,x,a)
Simplify / expand / factor	simplify(f), expand(f), factor(f)
Solve	solve(eq,x)
Derivative	diff(f,x)
Integral	int(f,x) / int(f,x,a,b)
Limit	limit(f,x,a)
Taylor	taylor(f,x,a,...)
Symbolic plot	fplot(f)
Numerical plot	plot(x,y)
Row / column vector	[1 2 3] / [1;2;3]
Sequence	start:step:end
Matrix multiplication	A*B

Task	Syntax
Element-wise multiplication	A.*B
Element-wise power	A.^2
Display / formatted output	disp(x) / fprintf(...)

Final principle: Do not memorize MATLAB as a collection of commands. Learn the mapping between mathematical notation and MATLAB syntax. Once that mapping is strong, the later Mathematics-I units become much easier to implement.